LAPORAN PRAKTIKUM

Struktur Data

“Double Linked List”

DISUSUN OLEH:

ABDUL JABBAR

2511533021

DOSEN PENGAMPU:

Dr. WAHYUDI, S.T, M.T

ASISTEN PRAKTIKUM:

SHERLY SUKMADIRA

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, penulis dapat menyelesaikan laporan/makalah mengenai Double Linked List ini dengan baik dan tepat waktu.

Laporan ini disusun sebagai salah satu bentuk pembelajaran dan pemahaman mengenai struktur data, khususnya pada konsep Double Linked List dalam pemrograman. Double Linked List merupakan salah satu jenis struktur data linear yang memiliki dua arah hubungan antar node, yaitu pointer menuju node berikutnya (*next*) dan node sebelumnya (*previous*). Struktur data ini sangat penting dipelajari karena dapat mempermudah proses manipulasi data seperti penyisipan, penghapusan, dan penelusuran data secara lebih fleksibel dibandingkan Single Linked List.

Dalam penyusunan laporan ini, penulis memperoleh banyak pengetahuan dan pengalaman baru terkait cara kerja, implementasi, serta penerapan Double Linked List dalam bahasa pemrograman. Penulis juga menyadari bahwa laporan ini masih memiliki kekurangan, baik dari segi isi maupun penyajiannya. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun demi perbaikan laporan di masa mendatang.

Penulis mengucapkan terima kasih kepada dosen, teman-teman, dan semua pihak yang telah memberikan dukungan serta bantuan dalam penyusunan laporan ini. Semoga laporan ini dapat memberikan manfaat dan menambah wawasan bagi para pembaca mengenai struktur data Double Linked List.

Padang,12 Mei 2026

Abdul Jabbar

DAFTAR ISI

KATA PENGANTAR ii
BAB I PENDAHULUAN 1
1.1 Latar Belakang 1
1.2 Tujuan 1
1.3 Manfaat 2
BAB II PEMBAHASAN 3
2.1 Double Linked List 3
2.2 Dasar Teori 3
2.3 Perbedaan single linked list dan double linked list 3
2.4 Praktikum pekan 6 4
2.4.1 class NodeDLL_2511533021 4
2.4.2 class InsertDLL_2511533021 6
2.4.3 class penelusuran_2511533021 9
2.4.4 class HapusDLL_2511533021 11
3.1 Kesimpulan 17
3.2 Saran 17
DAFTAR PUSTAKA 19

BAB I
PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi dan pemrograman saat ini menuntut adanya pengolahan data yang cepat, efisien, dan terstruktur. Dalam dunia pemrograman, struktur data menjadi salah satu konsep penting yang digunakan untuk mengatur dan menyimpan data agar dapat diakses serta dikelola dengan mudah. Salah satu struktur data yang sering digunakan adalah Linked List. Linked List merupakan struktur data dinamis yang terdiri dari kumpulan node yang saling terhubung menggunakan pointer atau referensi.

Seiring berkembangnya kebutuhan pengolahan data, penggunaan Single Linked List terkadang memiliki keterbatasan, terutama dalam proses penelusuran data yang hanya dapat dilakukan dalam satu arah. Oleh karena itu, dikembangkanlah Double Linked List yang memiliki dua buah pointer pada setiap node, yaitu pointer menuju node berikutnya (*next*) dan pointer menuju node sebelumnya (*previous*). Dengan adanya dua arah hubungan tersebut, proses traversing, penyisipan, dan penghapusan data menjadi lebih fleksibel dan efisien.

Double Linked List banyak diterapkan dalam berbagai sistem komputer dan aplikasi, seperti navigasi halaman pada browser, fitur undo dan redo pada aplikasi, manajemen playlist musik, serta pengelolaan data lainnya yang membutuhkan akses dua arah. Pemahaman mengenai konsep Double Linked List sangat penting bagi mahasiswa dan programmer karena struktur data ini menjadi dasar dalam pengembangan algoritma dan aplikasi yang lebih kompleks.

1.2 Tujuan

1. Memahami konsep Double Linked List.
2. Mengetahui cara membuat sebuah Node pada Double Linked List.

3. Mampu membuat Double Linked List sendiri.
4. Memahami operasi-operasi yang terdapat pada Double Linked List.

1.3 Manfaat

1. Membantu memahami cara pengelolaan data yang terhubung dua arah menggunakan pointer *next* dan *previous*.
2. Melatih kemampuan logika dan pemrograman dalam mengimplementasikan struktur data pada suatu program.
3. Mempermudah proses pengolahan data seperti penambahan, penghapusan, dan penelusuran data secara lebih fleksibel.
4. Menjadi dasar pengetahuan dalam mempelajari struktur data dan algoritma yang lebih kompleks.
5. Membantu mahasiswa dalam menyelesaikan permasalahan pemrograman yang membutuhkan akses data dua arah.
6. Menambah wawasan mengenai penerapan Double Linked List dalam berbagai aplikasi komputer, seperti navigasi browser, playlist musik, dan fitur undo-redo.

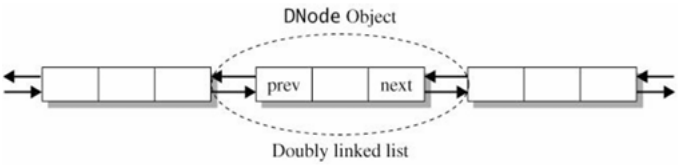
BAB II
PEMBAHASAN

2.1 Double Linked List

Double linked list adalah sebuah struktur data yang terdiri dari serangkaian node yang saling terhubung dengan dua pointer, yaitu pointer prev ke elemen sebelumnya dan pointer next ke elemen berikutnya. Setiap node memiliki tiga komponen, yaitu data, pointer prev, dan pointer next. Pointer prev menunjuk ke node sebelumnya dalam daftar, sedangkan pointer next menunjuk ke node berikutnya. Struktur ini memungkinkan double linked list untuk menyimpan dan mengakses data secara efisien dari kedua arah.

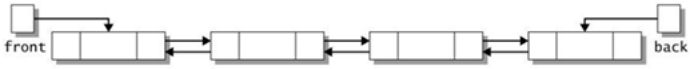
2.2 Dasar Teori

Double Linked List terdiri dari tiga bagian yaitu untuk menyimpan nilai dan dua reference yang menunjuk ke node selanjutnya (next node) dan node sebelumnya (previous node). Untuk bergerak maju dan mundur pada double linked list menggunakan link next dan prev pada node.



Gambar 2.1 *Representasi Sebuah Node di Double Linked List*

Double Linked List mempunyai reference front untuk menandai awal node dan reference back untuk menandai akhir list



Gambar 2.2 *Double Linked List*

2.3 Perbedaan single linked list dan double linked list

Single linked list hanya memiliki satu pointer next, sedangkan double linked list memiliki dua pointer, prev dan next.

kelebihan menggunakan double linked list:

- Akses Dua Arah**:** Double linked list memungkinkan akses data secara efisien dari kedua arah, baik dari awal maupun dari akhir.
- Insersi dan Penghapusan Cepat: Operasi penyisipan dan penghapusan elemen dapat dilakukan dengan cepat karena tidak memerlukan penggeseran elemen-elemen lainnya.
- Navigasi Efisien: Pointer prev dan next memungkinkan navigasi daftar secara efisien tanpa perlu melintasi seluruh daftar.

2.4 Praktikum pekan 6

Pada praktikum Pekan 6, mahasiswa Informatika Universitas Andalas mempelajari konsep Double Linked List (DLL) dengan membuat dan mengimplementasikan kode program menggunakan aplikasi Eclipse IDE.

2.4.1 class NodeDLL_2511533021

```
package pekan6_2511533021;
public class NodeDLL_2511533021 {
int data_3021;
NodeDLL_2511533021 next_3021;
NodeDLL_2511533021 prev_3021;

public NodeDLL_2511533021(int data_3021) {
    this.data_3021 = data_3021;
    this.next_3021 = null;
    this.prev_3021 = null;
}
}
```

Gambar 2.3 *Kode program*

1. Deklarasi Atribut

   - data_3021 : menyimpan nilai data pada node
   - next_3021 : menunjuk ke node berikutnya
   - prev_3021 : menunjuk ke node sebelumnya

2. Konsep Double Linked List

Setiap node memiliki dua pointer, yaitu next untuk menuju node berikutnya dan prev untuk kembali ke node sebelumnya.

3. ConstructorConstructor

   digunakan untuk menginisialisasi data node saat objek dibuat.

4. Inisialisasi Data

   this.data_3021 = data_3021 digunakan untuk mengisi nilai data

5. Inisialisasi Pointer

   - next_3021 diatur null karena belum terhubung ke node berikutnya
   - prev_3021 diatur null karena belum terhubung ke node sebelumnya

6. Fungsi Class

   Class ini berfungsi sebagai dasar pembentukan Double Linked List.

2.4.2 class InsertDLL_2511533021

```java
package pekan6_2511533021;

public class InsertDLL_2511533021 {

    static NodeDLL_2511533021 insertBegin(NodeDLL_2511533021 head, int data) {

        NodeDLL_2511533021 new_node = new NodeDLL_2511533021(data);

        new_node.next_3021 = head;

        if (head != null) {
            head.prev_3021 = new_node;
        }

        return new_node;
    }

    public static NodeDLL_2511533021 insertEnd(NodeDLL_2511533021 head, int newData) {

        NodeDLL_2511533021 newNode = new NodeDLL_2511533021(newData);

        if (head == null) {
            head = newNode;
        } else {

            NodeDLL_2511533021 curr = head;

            while (curr.next_3021 != null) {
                curr = curr.next_3021;
            }

            curr.next_3021 = newNode;
            newNode.prev_3021 = curr;
        }

        return head;
    }

    public static NodeDLL_2511533021 insertAtPosition(NodeDLL_2511533021 head, int pos, int new_data) {

        NodeDLL_2511533021 new_node = new NodeDLL_2511533021(new_data);

        if (pos == 1) {

            new_node.next_3021 = head;

            if (head != null) {
                head.prev_3021 = new_node;
            }

            head = new_node;
            return head;
        }

        NodeDLL_2511533021 curr = head;

        for (int i = 1; i < pos - 1 && curr != null; ++i) {
            curr = curr.next_3021;
        }

        if (curr == null) {
            System.out.println("posisi tidak ada");
            return head;
        }

        new_node.prev_3021 = curr;
        new_node.next_3021 = curr.next_3021;
        curr.next_3021 = new_node;

        if (new_node.next_3021 != null) {
            new_node.next_3021.prev_3021 = new_node;
        }

        return head;
    }

    public static void printList(NodeDLL_2511533021 head) {

        NodeDLL_2511533021 curr = head;

        while (curr != null) {
            System.out.print(curr.data_3021 + "<->");
            curr = curr.next_3021;
        }

        System.out.println();
    }

    public static void main(String[] args) {

        NodeDLL_2511533021 head = new NodeDLL_2511533021(2);
```

```java
        head.next_3021 = new NodeDLL_2511533021(3);
        head.next_3021.prev_3021 = head;

        head.next_3021.next_3021 = new NodeDLL_2511533021(5);
        head.next_3021.next_3021.prev_3021 = head.next_3021;

        System.out.println("DLL Awal: ");
        printList(head);

        head = insertBegin(head, 1);

        System.out.print("simpul 1 di tambah di awal : ");
        printList(head);

        System.out.print("simpul 6 di tambah di akhir: ");

        int data = 6;
        head = insertEnd(head, data);

        printList(head);

        System.out.println("tambah node 4 di posisi 4: ");

        int data2 = 4;
        int pos = 4;

        head = insertAtPosition(head, pos, data2);

        printList(head);
    }
}
```

Gambar 2.4

1. Method insertBegin

   Digunakan untuk menambahkan node di bagian awal Double Linked List.

2. Node baru diarahkan ke head lama.

   Jika head tidak null, maka prev dari head lama diarahkan ke node baru.Node baru menjadi head baru.

3. Method insertEnd

   Digunakan untuk menambahkan node di bagian akhir Double Linked List. Jika list kosong, node baru menjadi head.Jika tidak, program menelusuri sampai node terakhir lalu menambahkan node baru di akhir.Pointer prev node baru diarahkan ke node sebelumnya.

4. Method insertAtPosition

   Digunakan untuk menambahkan node pada posisi tertentu. Jika posisi 1, maka node ditambahkan di awal.Jika posisi lain, program menelusuri node sampai posisi yang diinginkan lalu menyisipkan node baru.Pointer next dan prev diperbarui agar hubungan antar node tetap benar.

5. Method printList

   Digunakan untuk menampilkan seluruh isi Double Linked List. Data ditampilkan dengan format <-> antar node.

6. Method main

   Merupakan titik awal eksekusi program.

7. Pembuatan Double Linked List Awal

   Membuat linked list dengan data 2, 3, dan 5.Setiap node saling terhubung menggunakan pointer next dan prev.

8. Menampilkan DLL Awal

   Menampilkan isi Double Linked List sebelum dilakukan penambahan.

9. Penambahan di Awal

   Menambahkan node bernilai 1 di bagian awal linked list.Penambahan di Akhir.Menambahkan node bernilai 6 di bagian akhir linked list.Penambahan di Posisi TertentuMenambahkan node bernilai 4 pada posisi ke-4

2.4.3 class penelusuran_2511533021

```java
package pekan6_2511533021;

public class penelusuran_2511533021 {

    static void forwardTraversal(NodeDLL_2511533021 head) {

        NodeDLL_2511533021 curr_3021 = head;

        while (curr_3021 != null) {
            System.out.print(curr_3021.data_3021 + "<->");
            curr_3021 = curr_3021.next_3021;
        }

        System.out.println();
    }

    static void backwardTraversal(NodeDLL_2511533021 tail) {

        NodeDLL_2511533021 curr_3021 = tail;

        while (curr_3021 != null) {
            System.out.print(curr_3021.data_3021 + "<->");
            curr_3021 = curr_3021.prev_3021;
        }

        System.out.println();
    }

    public static void main(String[] args) {

        NodeDLL_2511533021 head = new NodeDLL_2511533021(1);
```

```java
        NodeDLL_2511533021 second = new NodeDLL_2511533021(2);
        NodeDLL_2511533021 third = new NodeDLL_2511533021(3);

        head.next_3021 = second;
        second.prev_3021 = head;

        second.next_3021 = third;
        third.prev_3021 = second;

        System.out.println("Penelusuran maju: ");
        forwardTraversal(head);

        System.out.println("Penelusuran mundur: ");
        backwardTraversal(third);
    }
}
```

Gambar 2.5

1. Method forwardTraversal

   Digunakan untuk menelusuri node dari depan ke belakang. Penelusuran dimulai dari head dan bergerak menggunakan pointer next_3021.

2. Proses Penelusuran Maju

   Selama node tidak null, data pada node ditampilkan.Pointer curr_3021 berpindah ke node berikutnya menggunakan next_3021.

3. Method backwardTraversal

Digunakan untuk menelusuri node dari belakang ke depan. Penelusuran dimulai dari tail dan bergerak menggunakan pointer prev_3021.

4. Proses Penelusuran Mundur

   Selama node tidak null, data pada node ditampilkan.
   Pointer curr_3021 berpindah ke node sebelumnya menggunakan prev_3021.

5. Method main

   Merupakan titik awal eksekusi program.

6. Pembuatan Double Linked List

   Membuat tiga node dengan data 1, 2,dan 3.
   Setiap node saling terhubung menggunakan pointer next dan prev.

7. Menghubungkan Node

   - head.next_3021 diarahkan ke second
   - second.prev_3021 diarahkan ke head
   - second.next_3021 diarahkan ke third
   - third.prev_3021 diarahkan ke second

8. Penelusuran Maju

   Method forwardTraversal(head) digunakan untuk menampilkan data dari awal ke akhir.

9. Penelusuran Mundur

Method backwardTraversal(third) digunakan untuk menampilkan data dari akhir ke awal.

2.4.4 class HapusDLL_2511533021

```java
package pekan6_2511533021;

import pekan6_2511533021.NodeDLL_2511533021;

public class HapusDLL_2511533021 {

    public static NodeDLL_2511533021 delHead(NodeDLL_2511533021 head) {

        if (head == null) {
            return null;
        }

        NodeDLL_2511533021 temp = head;
        head = head.next_3021;

        if (head != null) {
            head.prev_3021 = null;
        }

        return head;
    }

    public static NodeDLL_2511533021 delLast(NodeDLL_2511533021 head) {

        if (head == null) {
            return null;
        }

        if (head.next_3021 == null) {
            return null;
        }
```

```java
        NodeDLL_2511533021 curr = head;

        while (curr.next_3021 != null) {
            curr = curr.next_3021;
        }

        if (curr.prev_3021 != null) {
            curr.prev_3021.next_3021 = null;
        }

        return head;
    }

    public static NodeDLL_2511533021 delPos(NodeDLL_2511533021 head, int pos) {

        if (head == null) {
            return head;
        }

        NodeDLL_2511533021 curr = head;

        for (int i = 1; curr != null && i < pos; ++i) {
            curr = curr.next_3021;
        }

        if (curr == null) {
            return head;
        }

        if (curr.prev_3021 != null) {
            curr.prev_3021.next_3021 = curr.next_3021;
```

```java
        }

        if (curr.next_3021 != null) {
            curr.next_3021.prev_3021 = curr.prev_3021;
        }

        if (head == curr) {
            head = curr.next_3021;
        }

        return head;
    }

    public static void printList(NodeDLL_2511533021 head) {

        NodeDLL_2511533021 curr = head;

        while (curr != null) {
            System.out.print(curr.data_3021 + " <-> ");
            curr = curr.next_3021;
        }

        System.out.println();
    }
    public static void main(String[] args) {
        //buat sebuah DLL
        NodeDLL_2511533021 head_3021 = new NodeDLL_2511533021(1);
        head_3021.next_3021 = new NodeDLL_2511533021(2);
        head_3021.next_3021.prev_3021 = head_3021;
        head_3021.next_3021.next_3021 = new NodeDLL_2511533021(3);
        head_3021.next_3021.next_3021.prev_3021 = head_3021.next_3021;
        head_3021.next_3021.next_3021.next_3021 = new NodeDLL_2511533021(4);
        head_3021.next_3021.next_3021.next_3021.prev_3021 = head_3021.next_3021.next_3021;
        head_3021.next_3021.next_3021.next_3021.next_3021 = new NodeDLL_2511533021(5);
        head_3021.next_3021.next_3021.next_3021.next_3021.prev_3021 = head_3021.next_3021.next_3021.next_3021;
        System.out.print("Dll Awal : ");
        printList(head_3021);

        System.out.print("Setelah head dihapus : ");
        head_3021 = delHead(head_3021);
        printList(head_3021);

        System.out.print("Setelah node terakhir dihapus : ");
        head_3021 = delLast(head_3021);
        printList(head_3021);

        System.out.print("menghapus node ke 2 : ");
        head_3021 = delPos(head_3021, 2);
        printList(head_3021);
    }
}
```

Gambar 2.7

1. Package

   package pekan6_2511533021 digunakan untuk mengelompokkan class agar program lebih terstruktur.
2. Import Class import pekan6_2511533021.NodeDLL_2511533021 digunakan untuk memanggil class node pada Double Linked List.
3. Deklarasi Class HapusDLL_2511533021 Class ini digunakan untuk melakukan operasi penghapusan node pada Double Linked List.
4. Method delHead Digunakan untuk menghapus node pertama (head).Jika list kosong, method mengembalikan null.Jika tidak, head dipindahkan ke node berikutnya dan prev dari head baru diatur menjadi null.
5. Method delLast Digunakan untuk menghapus node terakhir.Jika list kosong atau hanya memiliki satu node, methodmengembalikan null.Program menelusuri hingga node terakhir lalu memutus hubungan node sebelumnya dengan node terakhir.

6. Method delPos Digunakan untuk menghapus node pada posisi tertentu.
7. Program menelusuri node sampai posisi yang diminta.Pointer next dan prev diperbarui agar hubungan antar node tetap benar setelah penghapusan.
8. Method printList Digunakan untuk menampilkan seluruh isi Double Linked List.Data ditampilkan dengan format <-> antar node.Method mainMerupakan titik awal eksekusi program.
9. Pembuatan Double Linked List Membuat Double Linked List dengan data 1, 2, 3, 4, dan 5. Setiap node dihubungkan menggunakan pointer next dan prev.
10. Menampilkan DLL Awal Menampilkan isi linked list sebelum dilakukan penghapusan.
11. Penghapusan Head Menghapus node pertama dari Double Linked List menggunakan method delHead.Penghapusan Node Terakhir Menghapus node terakhir menggunakan method delLast.Penghapusan Berdasarkan Posisi Menghapus node pada posisi ke-2 menggunakan method delPos.Konsep Double Linked List

2.4.5 Laporan Tugas

```java
// Kelas Lagu_3021
package pekan6_2511533021;

public class Lagu_2511533021 {

    // atribut lagu
    String judul_3021;
    String penyanyi_3021;

    // pointer next dan prev
    Lagu_2511533021 next_3021;
    Lagu_2511533021 prev_3021;

    // constructor
    public Lagu_2511533021(String judul_3021, String penyanyi_3021) {
        this.judul_3021 = judul_3021;
        this.penyanyi_3021 = penyanyi_3021;
        this.next_3021 = null;
        this.prev_3021 = null;
    }

    // getter judul
    public String getJudul_3021() {
        return judul_3021;
    }

    // setter judul
    public void setJudul_3021(String judul_3021) {
        this.judul_3021 = judul_3021;
    }

    // getter penyanyi
    public String getPenyanyi_3021() {
        return penyanyi_3021;
    }

    // setter penyanyi
    public void setPenyanyi_3021(String penyanyi_3021) {
        this.penyanyi_3021 = penyanyi_3021;
    }
}
```

Package pekan6_2511533021 digunakan untuk mengelompokkan class agar program lebih terstruktur dan mudah dikelola. Pada program ini terdapat class Lagu_2511533021 yang berfungsi untuk merepresentasikan data lagu pada struktur data Double Linked List. Class tersebut memiliki beberapa atribut, yaitu judul_3021 untuk menyimpan judul lagu, penyanyi_3021 untuk menyimpan nama penyanyi, next_3021 sebagai pointer yang menunjuk ke node berikutnya, dan prev_3021 sebagai pointer yang menunjuk ke node sebelumnya. Konsep Double Linked List memungkinkan setiap node memiliki dua hubungan sehingga data dapat diakses dari arah depan maupun belakang.

Constructor pada class digunakan untuk menginisialisasi data lagu saat objek dibuat. Nilai judul_3021 diisi dengan judul lagu, sedangkan penyanyi_3021 diisi dengan nama penyanyi. Selain itu, pointer next_3021 dan prev_3021 diatur bernilai null karena node belum terhubung dengan node lain. Program ini juga memiliki method getter dan setter. Method getJudul_3021() digunakan untuk mengambil nilai judul lagu, sedangkan setJudul_3021() digunakan untuk mengubah judul lagu. Method getPenyanyi_3021() digunakan

untuk mengambil nama penyanyi, dan setPenyanyi_3021() digunakan untuk mengubah nama penyanyi.

```java
// Kelas Musik 3021
package pekan6_2511533021;

import java.util.Scanner;

public class Musik_2511533021 {

    // head dan tail
    Lagu_2511533021 head_3021 = null;
    Lagu_2511533021 tail_3021 = null;

    Scanner input_3021 = new Scanner(System.in);

    // method tambah lagu di akhir
    public void tambahLagu_3021() {

        System.out.print("Masukkan Judul Lagu : ");
        String judul_3021 = input_3021.nextLine();

        System.out.print("Masukkan Penyanyi : ");
        String penyanyi_3021 = input_3021.nextLine();

        // membuat node baru
        Lagu_2511533021 laguBaru_3021 =
                new Lagu_2511533021(judul_3021, penyanyi_3021);

        // jika playlist kosong
        if (head_3021 == null) {
            head_3021 = laguBaru_3021;
            tail_3021 = laguBaru_3021;
        }
```

Class Musik_2511533021 digunakan untuk mengelola playlist lagu menggunakan struktur data Double Linked List. Class ini memiliki head_3021 dan tail_3021 sebagai penunjuk node pertama dan terakhir pada playlist. Program menyediakan beberapa method utama, yaitu tambahLagu_3021() untuk menambahkan lagu di akhir playlist, hapusLaguAwal_3021() untuk menghapus lagu pertama, tampilMaju_3021() untuk menampilkan playlist dari awal ke akhir, tampilMundur_3021() untuk menampilkan playlist dari akhir ke awal menggunakan pointer prev, serta cariLagu_3021() untuk mencari lagu berdasarkan judul tanpa membedakan huruf besar dan kecil (*case insensitive*).

Pada method main, program menampilkan menu interaktif menggunakan Scanner sehingga pengguna dapat memilih operasi yang diinginkan, seperti menambah lagu, menghapus lagu, melihat playlist, mencari lagu, atau keluar dari program. Program ini menunjukkan penerapan Double Linked List dalam pengelolaan playlist lagu secara sederhana dan terstruktur.

```java
        // jika sudah ada isi
        else {
            tail_3021.next_3021 = laguBaru_3021;
            laguBaru_3021.prev_3021 = tail_3021;
            tail_3021 = laguBaru_3021;
        }

        System.out.println("Lagu berhasil ditambahkan!");
    }

    // method hapus lagu awal
    public void hapusLaguAwal_3021() {

        // jika kosong
        if (head_3021 == null) {
            System.out.println("Playlist kosong!");
            return;
        }

        // jika hanya 1 lagu
        if (head_3021 == tail_3021) {
            head_3021 = null;
            tail_3021 = null;
        }

        // jika lebih dari 1 lagu
        else {
            head_3021 = head_3021.next_3021;
            head_3021.prev_3021 = null;
        }
```

```java
        System.out.println("Lagu pertama berhasil dihapus!");
    }

    // tampil playlist maju
    public void tampilMaju_3021() {

        // jika kosong
        if (head_3021 == null) {
            System.out.println("Playlist kosong!");
            return;
        }

        Lagu_2511533021 bantu_3021 = head_3021;

        System.out.println("\n=== Playlist Maju ===");

        while (bantu_3021 != null) {

            System.out.println("Judul    : "
                    + bantu_3021.judul_3021);

            System.out.println("Penyanyi : "
                    + bantu_3021.penyanyi_3021);

            System.out.println("--------------------");

            bantu_3021 = bantu_3021.next_3021;
        }
    }

    // tampil playlist mundur
```

```
=== Playlist Musik NIM: 2511533021 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan : 1
Masukkan Judul Lagu : Rosalía - Mio Cristo Piange Diamanti
Masukkan Penyanyi : : Rosalía
Lagu berhasil ditambahkan!
```

BAB III
PEMBAHASAN

3.1 Kesimpulan

Berdasarkan pembahasan yang telah dilakukan, dapat disimpulkan bahwa Double Linked List merupakan salah satu struktur data dinamis yang terdiri dari kumpulan node yang saling terhubung dengan dua arah, yaitu pointer menuju node berikutnya (*next*) dan node sebelumnya (*previous*). Struktur data ini memungkinkan proses penelusuran data dilakukan secara maju maupun mundur sehingga lebih fleksibel dibandingkan Single Linked List.

Double Linked List memiliki berbagai operasi penting seperti penambahan, penghapusan, pencarian, dan penelusuran data yang dapat dilakukan dengan lebih efisien pada kondisi tertentu. Struktur data ini juga banyak digunakan dalam berbagai aplikasi komputer, seperti navigasi browser, playlist musik, dan fitur undo-redo.

Melalui praktikum dan pembelajaran mengenai Double Linked List, mahasiswa dapat memahami konsep dasar struktur data dinamis serta mampu mengimplementasikannya dalam bahasa pemrograman. Dengan demikian, pemahaman mengenai Double Linked List dapat menjadi dasar yang penting dalam mempelajari algoritma dan pengembangan program yang lebih kompleks.

3.2 Saran

Dalam mempelajari Double Linked List, mahasiswa disarankan untuk lebih sering berlatih membuat dan mengimplementasikan program secara langsung agar dapat memahami konsep kerja node dan pointer dengan baik. Selain itu, mahasiswa juga perlu memahami alur proses penambahan, penghapusan, dan penelusuran data agar tidak terjadi kesalahan dalam pengelolaan struktur data.

Penggunaan aplikasi pemrograman seperti Eclipse IDE dapat membantu mahasiswa dalam menulis, menjalankan, dan memperbaiki kode program dengan lebih mudah. Mahasiswa juga disarankan untuk mempelajari penerapan Double Linked List dalam berbagai kasus nyata sehingga pemahaman yang diperoleh tidak hanya bersifat teori, tetapi juga dapat diterapkan dalam pengembangan program dan aplikasi komputer.

DAFTAR PUSTAKA

Arna Fariza. *Praktikum 17 – Double Linked List 1*. Politeknik Elektronika Negeri Surabaya (PENS). Diakses pada Mei 2026. Microsoft Word - Praktikum 17 - Double Linked List 1

Definisi.ac.id. *Double Linked List*. Diakses pada Mei 2026. Double Linked List: Panduan Lengkap untuk Pemula - Definisi